```
    {
        var conn = Configuration.GetConnectionString("DefaultConnection");

        services.AddDbContext<AppDbContext>(options =>              ❸
            options.UseSqlite(conn));                               ❸
        services.AddDefaultIdentity<ApplicationUser>(options =>     ❸
            options.SignIn.RequireConfirmedAccount = true)          ❸
            .AddEntityFrameworkStores<AppDbContext>();              ❸

        services.AddScoped<RecipeService>();                        ❹
        services.AddRazorPages();                                   ❺

        services.AddScoped<IAuthorizationHandler, IsRecipeOwnerHandler>();
        services.AddAuthorizationBuilder()
            .AddPolicy("CanManageRecipe",
                p => p.AddRequirements(new IsRecipeOwnerRequirement()));
    }

    public void Configure(IApplicationBuilder app) => { /* Not shown */ }
}
```

❶ The IConfiguration for the app is injected into the constructor.

❷ You must register your services against the provided IServiceCollection.

❸ Registers all the EF Core and ASP.NET Core Identity services

❹ Registers the custom service implementations

❺ Registers the framework services

After configuring all your services, you need to set up your middleware pipeline and map your endpoints. The process is similar to configuring your middleware pipeline using `WebApplication`:

- You add middleware to the pipeline by calling `Use*` extension methods on an `IApplicationBuilder` instance.
- The order in which you add the middleware to the pipeline is important and defines the final pipeline order.
- You can add middleware conditionally based on the environment.